

# Execution Manual

This repository contains the computational workflow used to generate the analyses, figures, and tables reported in the accompanying manuscript: package source, per-stage CLI scripts, Snakemake orchestration, environment specifications, and tests.

## Quick start

```
tools/run_pipeline.sh
```

This one command works on any Linux host, from a bare `git clone`, with no manual environment preparation. It tries, in order, the first usable route (Docker, then Apptainer, then native conda/Snakemake), prepares that route's environment automatically, and runs the complete workflow through to `computational_analysis_complete`. All seven required datasets download and checksum-verify automatically. Expect on the order of hours and roughly 100 GB of disk; see "Computational requirements".

To select a specific route instead of auto-probing:

```
tools/run_pipeline.sh --backend docker|apptainer|native snakemake --cores 8
```

See "Running the pipeline" below for per-route detail.

## Datasets

Seven datasets are used, all acquired automatically as part of the default `snakemake --cores N` target. Full accession/path/checksum detail is in `manifests/dataset_registry.yaml`. Each is a public, unauthenticated download (NCBI GEO, 10x Genomics' public CDN, or UCSC Xena). No login, API key, or manual step is needed for any of them.

All paths below are relative to `data/`.

| Dataset                             | Accession   | Location                          | Size     |
|-------------------------------------|-------------|-----------------------------------|----------|
| Primary Xenium HNSCC cohort         | GSE300147   | raw/                              | ~53 GiB  |
| Companion scRNA-seq + scTCR-seq VDJ | GSE287301   | external /GSE287301/              | ~5.5 GB  |
| Puram et al. 2017 scRNA-seq         | GSE103322   | external /GSE103322/              | ~90 MB   |
| Cillo et al. 2020 scRNA-seq         | GSE139324   | external/scrna /GSE139324/        | ~1.1 GB* |
| TCGA-HNSC bulk RNA-seq              | TCGA-HNSC   | external/bulk /TCGA-HNSC/         | ~62 MB   |
| Janesick 2023 Xenium breast         | GSE243280** | external/spatial / (Janesick dir) | ~323 MB* |
| de Oliveira 2025 Xenium colorectal  | GSE280314   | external/spatial / (Oliveira dir) | ~323 MB* |

\* Shared with a sibling dataset in the same parent directory. \*\* Xenium data specifically is under SubSeries GSE243168.

Every acquisition script downloads only if not already present, then verifies a SHA-256 checksum (checksums . sha256, tracked alongside each dataset's README .md under data/external/, which also records the exact source URL, acquisition date, citation, and licence).

## Computational requirements

| Resource          | Requirement                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| OS / architecture | Linux, x86_64/amd64. Not tested on macOS, Windows, or arm64.                                                             |
| CPU               | CPU-only, no GPU used. Runs on anything from 1 core up; more cores reduce wall-clock time via --cores N.                 |
| Memory            | 32 GB recommended (heavier steps such as Leiden clustering over ~1.1M cells need this).                                  |
| Disk              | ~60 GB for all seven datasets, ~100 GB including all generated outputs. 150 GB+ free recommended.                        |
| Network           | Needed once, up front, for cloning, environment resolution, and dataset downloads (~60 GB total). Not needed after that. |
| Runtime           | ~5.5 hours for the full default target on an 8-core machine; varies with hardware.                                       |

tools/run\_pipeline.sh creates the conda environment and builds or pulls the Docker/Apptainer image automatically. It cannot install Docker or Apptainer themselves, or grant the host-level permissions each needs: a running Docker daemon with docker group membership (or rootless Docker); for Apptainer, a central installation and, only if building rather than running an existing image, root/- -fakeroot support. These are one-time host prerequisites, not part of the scientific analysis.

## Repository contents

|                           |                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------|
| src/xenium_tcr_ecology/   | installable package: reusable, tested analysis logic                                 |
| scripts/<NN_stage>/       | thin CLI entry points, one subfolder per stage                                       |
| workflow/rules/           | Snakemake rule definitions, one file per stage                                       |
| governance/               | analysis registry, taxonomy version log, freeze decisions, HPV contrast registration |
| config/                   | thresholds, seeds, graph/interaction parameters, policy                              |
| manifests/                | dataset registry, project-root marker, phase registry                                |
| environment/, containers/ | conda spec + lock, Docker image                                                      |
| tests/                    | unit/, simulation/, r_unit/, fixtures/                                               |
| docs/                     | this manual, plus architecture/phase reference (mkdocs)                              |
| release/                  | installable software package, public data bundle (once built)                        |

data/, results/, reports/, figures/, tables/, and manuscript/ are gitignored except .gitkeep; see "Outputs" below.

## Running the pipeline

The default target (rule all) resolves to computational\_analysis\_ complete: raw data through statistical closure, figures, tables, and the rendered manuscript PDF.

```
.venv/bin/snakemake --cores 8 qc_and_integrity_release # run up to a specific checkpoint
```

Checkpoints, in execution order: `project_ready`, `raw_data_frozen`, `canonical_objects`, `qc_and_integrity_release`, `annotation_release`, `tumour_and_tcr_release`, `graph_and_calibration_release`, `niche_release`, `clone_descriptor_release`, `external_checkpoint`, `clone_model_release`, `interaction_release`, `hpv_release`, `full_validation_release`, `computational_analysis_complete`.

## Docker (preferred)

```
docker build -t xenium-tcr-ecology:local -f containers/Dockerfile .
docker run --rm --user "$(id -u):$(id -g)" -v "$(pwd):/workspace" \
  xenium-tcr-ecology:local \
  snakemake --cores 8
```

Or via the launcher, which does the above automatically and falls back to Apptainer/native if Docker is unusable:

```
tools/run_pipeline.sh snakemake --cores 8
```

`containers/Dockerfile` builds from the committed `environment/conda/environment.lock`, already present in a fresh clone. `--user "$(id -u):$(id -g)"` keeps files the container writes owned by the invoking user rather than root; the launcher applies this automatically.

## Apptainer/Singularity (fallback)

Used automatically when Docker is unavailable. `containers/Apptainer.def` builds from the same `environment/conda/environment.lock` as the other two routes. Tested with Apptainer 1.4.0.

```
apptainer build --fakeroot xenium-tcr-ecology.sif containers/Apptainer.def
apptainer exec --cleanenv --bind "$PWD:/workspace" --pwd /workspace \
  xenium-tcr-ecology.sif conda run -n xenium-tcr-ecology snakemake --cores 8
```

Or via the wrapper or unified launcher:

```
tools/run_with_apptainer.sh snakemake --cores 8
tools/run_pipeline.sh snakemake --cores 8
```

If Docker is available but `--fakeroot/root` is not, build via `docker build -f containers/Dockerfile -t xenium-tcr-ecology:latest .` then `apptainer build xenium-tcr-ecology.sif docker-daemon://xenium-tcr-ecology:latest` - this needs no privileged build step. Images built this way need `conda run -n xenium-tcr-ecology` rather than a bare `apptainer exec`, since they only expose the toolchain through the converted Docker entrypoint.

## Native (conda + Snakemake, final fallback)

Always fully supported, no container runtime needed:

```
git clone <this repository>
cd HNSCC-Xenium-Spatial-Ecology
mamba env create -f environment/conda/main.yml
conda activate xenium-tcr-ecology
pip install -e ".[dev]"
python3.13 -m venv .venv && .venv/bin/pip install snakemake
.venv/bin/snakemake --cores 8 -n # dry run
.venv/bin/snakemake --cores 8 # full run
```

Snakemake orchestration runs from this separate `.venv/` (Python 3.13), not the `xenium-tcr-ecology` conda env: the conda env provides the Python/R analysis stack every rule invokes, `.venv` provides only Snakemake itself. If

Python 3.13 isn't already available, install it via conda-forge (`mamba create -n py313_bootstrap python=3.13`) rather than the system package manager.

Stochastic procedures (permutation tests, bootstrap confidence intervals) read their seed centrally from `config/config.yaml`, producing deterministic output within the same already-built environment. This is not a claim of byte-identical output across different Oses, architectures, or BLAS/LAPACK implementations.

### Reproducibility notes

All three routes build from the identical `environment/conda/ environment.lock` and run the identical Snakefile, config, and rules - the route changes only how the software environment is prepared, never the science. `snakemake --cores N --forceall -n` resolves to the same 152 jobs under all three. The full test suite (708 Python via `pytest tests/unit tests/simulation`, 131 R via `testthat`) passes identically under all three.

A published, pre-built container image pulled by immutable digest is the intended path to byte-identical output across machines. All three routes currently depend on a local build or fresh conda resolve, which pins dependency versions but not necessarily identical build strings across machines.

### Stages

17 numbered stages under `scripts/`, one Snakemake rule file per stage under `workflow/rules/`. Full per-script detail (purpose, primary output, determinism) is generated from `manifests/phase_registry.yaml` into `docs/phases.md`; don't hand-edit that file directly.

| #  | Stage                                      | Purpose                                                          |
|----|--------------------------------------------|------------------------------------------------------------------|
| 01 | project_setup_and_governance               | Research governance, environment, and reproducibility setup      |
| 02 | raw_data_ingestion                         | Data acquisition, integrity verification, and staging            |
| 03 | spatialdata_import                         | Xenium import, coordinate harmonisation, canonical data objects  |
| 04 | quality_control                            | Multilevel QC, principled exclusion, transcript-integrity checks |
| 05 | preprocessing_and_normalisation            | Expression normalisation, feature engineering                    |
| 06 | cell_type_annotation                       | Hierarchical cell annotation with uncertainty                    |
| 07 | tumour_epithelium_characterisation         | Malignant-state inference, tumour-boundary reconstruction        |
| 08 | tcr_clonal_analysis                        | TCR probe decoding, clone assignment, validation                 |
| 09 | spatial_graph_construction_and_calibration | Spatial graphs, null-model calibration                           |

|    |                                                |                                                        |
|----|------------------------------------------------|--------------------------------------------------------|
| 10 | niche_and<br>_ecosystem<br>_discovery          | Discovery of spatial ecosystems and tissue domains     |
| 11 | clone_spatial<br>_descriptors                  | Clone-level ecological descriptors and structure test  |
| 12 | external<br>_checkpoint<br>_validation         | Early external sanity-check and taxonomy-freeze gate   |
| 13 | clone_ecology<br>_confirmatory<br>_models      | Confirmatory variance-partitioning models              |
| 14 | spatial<br>_interactions<br>_and_barriers      | Barrier topology and candidate spatial interactions    |
| 15 | hvp_stratified<br>_analysis                    | HPV-stratified analysis (consolidated, capped)         |
| 16 | external<br>_validation_and<br>_generalisation | External validation, triangulation, generalisability   |
| 17 | statistical<br>_closure_and<br>_release        | Statistical closure, figures, manuscript, release prep |

## Outputs

| Output                                     | Location                                      | Tier                                |
|--------------------------------------------|-----------------------------------------------|-------------------------------------|
| Main manuscript figures                    | figures/main/ (6)                             | Final                               |
| Supplementary figures                      | figures<br>/supplementary/ (23) + clone atlas | Final                               |
| Results tables                             | tables/ (10 .tsv)                             | Final                               |
| Per-module analysis reports                | reports/                                      | Intermediate                        |
| Per-module summary tables + run provenance | results/                                      | Intermediate                        |
| Frozen primary results                     | data/releases<br>/final_primary/              | Final, frozen (SHA-256 checksummed) |
| Rendered manuscript                        | manuscript<br>/manuscript.pdf                 | Final                               |
| Derived intermediate data                  | data/derived/                                 | Intermediate                        |

All of the above are gitignored. A fresh `git clone` includes no figure, table, or manuscript PDF; run the pipeline to produce them.

## Further reading

- `docs/phases.md` - auto-generated, full per-script stage reference.

- `docs/architecture.md` - package/pipeline structure and governance gates.
- `docs/analysis_amendments.md` - every analysis added after the initial pipeline was complete, with rationale and status.